TUGAS AKHIR

PENGEMBANGAN FITUR UJIAN DAN DOWNLOAD ASSIGNMENT PADA SHARIF-JUDGE

Christofer Setiawan

NPM: 6182301076

PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2026

FINAL PROJECT

DEVELOPMENT OF EXAM AND ASSIGNMENT DOWNLOAD FEATURES FOR SHARIF-JUDGE



Christofer Setiawan

NPM: 6182301076

DEPARTMENT OF INFORMATICS
FACULTY OF SCIENCES
PARAHYANGAN CATHOLIC UNIVERSITY
2026

ABSTRAK

«Tuliskan abstrak anda di sini, dalam bahasa Indonesia»

Kata-kata kunci: «Tuliskan di sini kata-kata kunci yang anda gunakan, dalam bahasa Indonesia»

ABSTRACT

«Tuliskan abstrak anda di sini, dalam bahasa Inggris»

Keywords: «Tuliskan di sini kata-kata kunci yang anda gunakan, dalam bahasa Inggris»

cfttoctitlename

cftloftitlename

BAB 1

PENDAHULUAN

1.1 Latar Belakang

SharIF-Judge merupakan *website* utama bagi mahasiswa informatika Universitas Katolik Parahyangan untuk berlatih dan mengasah kemampuan pemrograman. *Website* ini memuat berbagai *assignment* dan untuk masing-masing *assignment* memiliki *problem* yang harus diselesaikan mahasiswa sebagai bahan latihan. Saat ini SharIF-Judge melakukan penilaian secara otomatis dengan mencocokan keluaran kode dengan jawaban yang sudah disiapkan oleh pembuat soal dan memberikan nilai berdasarkan keakuratan keluaran yang dihasilkan oleh kode mahasiswa. Selain nilai yang diberikan oleh SharIF-Judge, dosen juga harus menilai kode secara manual untuk menilai logika dan efisiensi kode dari setiap mahasiswa. Namun evaluasi secara manual ini memiliki kendala tersendiri, dosen harus mengunduh dan memeriksa berkas jawaban mahasiswa satu per satu secara manual.

Selain kendala penilaian kode mahasiswa secara manual oleh dosen, terdapat pula beberapa kendala pada sisi mahasiswa. Saat ini SharIF-Judge memerlukan dua *instance* berbeda untuk mengerjakan latihan dan *quiz*. Pemisahan ini dilakukan agar mahasiswa tidak dapat melihat catatan dan hasil latihan yang sudah di kerjakan sebelumnya. Namun, pemisahan ini membuat *maintenance* menjadi kurang efisien dan cukup menyulitkan mahasiswa karena mengharuskan mengetik tautan secara manual untuk masuk ke SharIF-Judge khusus untuk *quiz*. Tidak hanya itu, tampilan antar muka SharIF-Judge saat ini masih minim penunjuk visual. Pada daftar *assignment* tidak ada penanda visual untuk membedakan latihan dan *quiz*, serta kurangnya indikator warna untuk durasi pengerjaan *assignment* yang membuat SharIF-Judge menjadi kurang interaktif.

Berdasarkan permasalahan yang dialami oleh dosen dan mahasiswa, pada tugas akhir ini akan mengembangkan dua fitur utama pada SharIF-Judge untuk membantu mengatasi kendala dosen dan mahasiswa. Fitur pertama menambahkan penanda status visual berupa *flag* dan menambahkan pembatasan akses berbasis IPv4, sehingga *quiz* dan latihan bisa tetap berada pada satu *instance* tanpa harus mengurangi keamanan. Akan dikembangkan pula pembaruan tampilan antarmuka dengan menambahkan emoji untuk membedakan latihan dengan *quiz* dan tiga indikator warna berbeda untuk durasi pengerjaan *assignment*. Fitur kedua yang akan dikembangkan adalah pengunduhan seluruh berkas *assignment* dalam format yang sudah terkompresi zip dengan penamaan file *{id-assignment}-{nama-assignment}-{nama-soal}.zip*.

Dengan adanya penggabungan dua *instance* SharIF-Judge menjadi satu *instance* dan tersedianya fitur mengunduh seluruh assignment secara bersamaan, diharapkan akan memudahkan dalam melakukan *maintenance* dan melakukan penilaian kode secara manual. Selain itu perubahan antarmuka SharIF-Judge diharapkan mampu meningkatkan kejelasan informasi bagi mahasiswa

saat mengerjakan latihan maupun *quiz*.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan pembatasan akses berdasarkan *IPv4* dan penanda *quiz* dalam satu *instance*?
2. Bagaimana merancang perubahan tampilan antarmuka agar memiliki status visual berupa emoji dan durasi pengerjaan setiap *assignment* diberikan warna sesuai dengan status *assignment*?
3. Bagaimana cara melakukan pengunduhan berkas *assignment* yang sudah dikompresi dalam bentuk zip dan memiliki penamaan *{id-assignment}-{nama-assignment}-{nama-soal}.zip*.

1.3 Tujuan

1. Mengimplementasikan pembatasan akses berdasarkan *IPv4* dan penanda *quiz* dalam satu *instance*.
2. Mengimplementasikan perubahan tampilan antarmuka agar memiliki status visual berupa emoji dan durasi pengerjaan setiap *assignment* diberikan warna sesuai dengan status *assignment*.
3. Mengimplementasikan pengunduhan berkas *assignment* yang sudah dikompresi menjadi bentuk zip dan memiliki penamaan *{id-assignment}-{nama-assignment}-{nama-soal}.zip*.

1.4 Batasan Masalah

Untuk mempermudah pembuatan template ini, tentu ada hal-hal yang harus dibatasi, misalnya saja bahwa template ini bukan berupa style LaTeX pada umumnya (dengan alasannya karena belum mampu jika diminta membuat seperti itu)

1.5 Metodologi

1. Melakukan studi mengenai pembatasan akses *website* berdasarkan IPv4.
2. Melakukan studi mengenai cara mengunduh banyak file secara bersamaan dan dijadikan format zip.
3. Merancang sistem untuk pembatasan akses berdasarkan IPv4 dan mengunduh banyak file lalu mengubahnya dalam bentuk zip.
4. Mengimplementasikan perubahan sistem pada *website* SharIF-Judge.
5. Melakukan Pengujian dan eksperimen.
6. Menganalisis hasil eksperimen.
7. Menulis dokumen tugas akhir.

1.6 Sistematika Pembahasan

- **Bab 1:** Pendahuluan
  Membahas latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.
- **Bab 2:** Landasan Teori
  Membahas teori-teori yang berhubungan dengan penelitian ini, yaitu SharIF-Judge, CodeIgniter 3, Twig, IDE, subnetting *IPv4*, mengunduh dan melakukan format file.
- **Bab 3:** Analisis
  Membahas analisis terhadap perangkat lunak SharIF-Judge dan IDE pada SharIF-Judge.
- **Bab 4:** Perancangan
  Membahas perancangan fitur yang akan diimplementasikan pada SharIF Judge.
- **Bab 5:** Implementasi dan Pengujian
  Membahas implementasi fitur pada SharIF Judge dan pengujian yang dilakukan beserta dengan analisis hasil pengujian.
- **Bab 6:** Kesimpulan dan Saran
  Membahas kesimpulan dari penelitian ini dan saran untuk penelitian berikutnya.

BAB 2

LANDASAN TEORI

2.1 Skripsi

Rencananya akan diisi dengan penjelasan umum mengenai buku skripsi.

2.2 LaTeX

Mengapa menggunakan LaTeX untuk buku skripsi dan apa keunggulan/kerugiannya bagi mahasiswa dan pembuat template.

2.3 Template Skripsi FTIS UNPAR

Akan dipaparkan bagaimana menggunakan template ini, termasuk petunjuk singkat membuat referensi, gambar dan tabel. Juga hal-hal lain yang belum terpikir sampai saat ini.

2.3.1 Tabel

Berikut adalah contoh pembuatan tabel. Penempatan tabel dan gambar secara umum diatur secara otomatis oleh LaTeX, perhatikan contoh di file bab2.tex untuk melihat bagaimana cara memaksa tabel ditempatkan sesuai keinginan kita.

Perhatikan bawa berbeda dengan penempatan judul gambar gambar, keterangan tabel harus diletakkan di atas tabel!! Lihat Tabel **??** berikut ini:

Tabel 2.1: Tabel contoh

	v_{start}	$\mathcal{S}_1$	v_{end}
τ_1	1	12	20
τ_2	1		20
τ_3	1	9	20
τ_4	1		20

Tabel **??** dan Tabel **??** berikut ini adalah tabel dengan sel yang berwarna dan ada dua tabel yang bersebelahan.

Tabel 2.2: Tabel bewarna(1)

	v_{start}	$\mathcal{S}_2$	$\mathcal{S}_1$	v_{end}
τ_1	1	5	12	20
τ_2	1	8		20
τ_3	1	2/8/17	9	20
τ_4	1			20

Tabel 2.3: Tabel bewarna(2)

	v_{start}	$\mathcal{S}_1$	$\mathcal{S}_2$	v_{end}
τ_1	1	12	5	20
τ_2	1		8	20
τ_3	1	9	2/8/17	20
τ_4	1			20

2.3.2 Kutipan

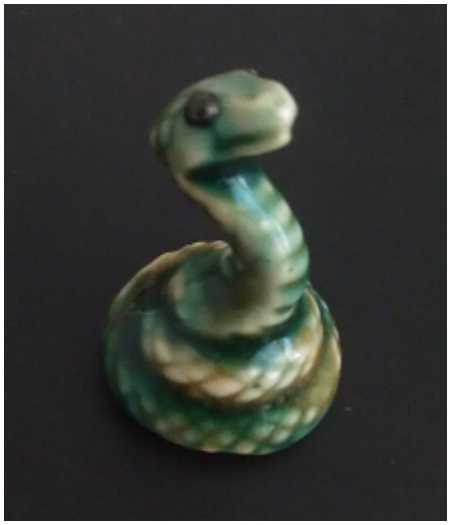
Berikut contoh kutipan dari berbagai sumber, untuk keterangan lebih lengkap, silahkan membaca file referensi.bib yang disediakan juga di template ini. Contoh kutipan:

- Buku: [?]
- Bab dalam buku: [?]
- Artikel dari Jurnal: [?]
- Artikel dari prosiding seminar/konferensi: [?]
- Skripsi/Thesis/Disertasi: [?] [?] [?]
- Technical/Scientific Report: [?]
- RFC (Request For Comments): [?]
- Technical Documentation/Technical Manual: [?] [?] [?]
- Paten: [?]
- Tidak dipublikasikan: [?] [?]
- Laman web: [?]
- Lain-lain: [?]

2.3.3 Gambar

Pada hampir semua editor, penempatan gambar di dalam dokumen LaTeX tidak dapat dilakukan melalui proses *drag and drop*. Perhatikan contoh pada file bab2.tex untuk melihat bagaimana cara menempatkan gambar. Beberapa hal yang harus diperhatikan pada saat menempatkan gambar:

- Setiap gambar **harus** diacu di dalam teks (gunakan *field* LABEL)
- *Field* CAPTION digunakan untuk teks pengantar pada gambar. Terdapat dua bagian yaitu yang ada di antara tanda [dan] dan yang ada di antara tanda { dan }. Yang pertama akan muncul di Daftar Gambar, sedangkan yang kedua akan muncul di teks pengantar gambar. Untuk skripsi ini, samakan isi keduanya.
- Jenis file yang dapat digunakan sebagai gambar cukup banyak, tetapi yang paling populer adalah tipe PNG (lihat Gambar **??**), tipe JPG (Gambar **??**) dan tipe PDF (Gambar **??**)
- Besarnya gambar dapat diatur dengan *field* SCALE.
- Penempatan gambar diatur menggunakan *placement specifier* (di antara tanda [dan] setelah deklarasi gambar. Yang umum digunakan adalah **H** untuk menempatkan gambar **sesuai** penempatannya di file .tex atau **h** yang berarti "kira-kira" di sini.
  Jika tidak menggunakan *placement specifier*, LaTeX akan menempatkan gambar secara otomatis untuk menghindari bagian kosong pada dokumen anda. Walaupun cara ini sangat mudah, hindarkan terjadinya penempatan dua gambar secara berurutan.

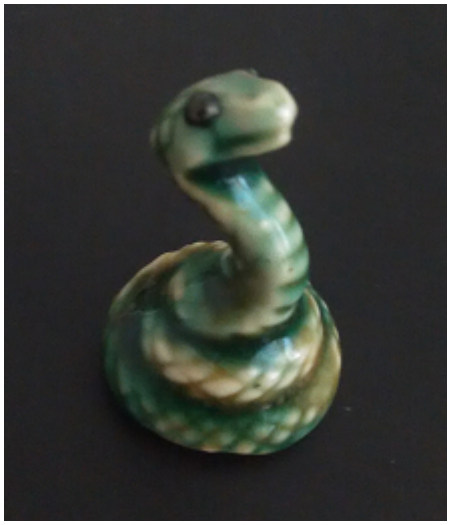


Gambar 2.1: Gambar *Serpentes* dalam format png

- Gambar **??** ditempatkan di bagian atas halaman, walaupun penempatannya dilakukan setelah penulisan 3 paragraf setelah penjelasan ini.
- Gambar **??** dengan skala 0.5 ditempatkan di antara dua buah paragraf. Perhatikan penulisannya di dalam file bab2.tex!
- Gambar **??** ditempatkan menggunakan *specifier* **h**.



Gambar 2.2: Ular kecil



Gambar 2.3: *Serpentes* jantan

2.3.4 Kode Program

Kode program dalam bahasa tertentu seringkali harus ditulis di dalam bab, bukan hanya dilampirkan di bagian Lampiran. Kode **??** menampilkan penggunaan karakter-karakter yang umum digunakan dalam sebuah program yang ditulis dengan bahasa C.

Kode 2.1: Kode untuk menampilkan karakter-karakter aneh

```
// This does not make algorithmic sense,
// but it shows off significant programming characters.

#include<stdio.h>

void myFunction( int input, float* output ) {
    switch ( array[i] ) {
        case 1: // This is silly code
            if ( a >= 0 || b <= 3 && c != x )
                *output += 0.005 + 20050;
            char = 'g';
            b = 2^n + ~right_size - leftSize * MAX_SIZE;
            c = (--aaa + &daa) / (bbb++ - ccc % 2 );
            strcpy(a,"hello $@?");
    }
    count = ~mask | 0x00FF00AA;
}

// Fonts for Displaying Program Code in LaTeX
// Adrian P. Robson, nepsweb.co.uk
// 8 October 2012
// http://nepsweb.co.uk/docs/progfonts.pdf
```

2.3.5 Notasi

Simbol-simbol (matematika) yang sering digunakan sepanjang penulisan skripsi, dapat dimasukkan ke dalam "Daftar Notasi". Daftar ini ada di halaman depan sebelum Bab **??**. Cara memasukkan sebuah simbol ke dalam Daftar Notasi adalah menggunakan perintah `\nomenclature`. Contoh:

```
\nomenclature[]{$A$}{luas kandang ular}
```

Argumen opsional digunakan untuk mengurutkan notasi. Silahkan lihat sendiri dokumentasi package `nomencl`

BAB 3

ANALISIS

3.1 Analisis Sistem Kini

3.1.1 Model, View, Controller

DAFTAR REFERENSI

[1] de Berg, M., Cheong, O., van Kreveld, M. J., dan Overmars, M. (2008) *Computational Geometry: Algorithms and Applications*, 3rd edition. Springer-Verlag, Berlin.

[2] van Kreveld, M. J. (2004) Geographic information systems. Bagian dari Goodman, J. E. dan O'Rourke, J. (ed.), *Handbook of Discrete and Computational Geometry*. Chapman & Hall/CRC, Boca Raton.

[3] Buchin, K., Buchin, M., van Kreveld, M. J., Löffler, M., Silveira, R. I., Wenk, C., dan Wiratma, L. (2013) Median trajectories. *Algorithmica*, **66**, 595–614.

[4] van Kreveld, M. J. dan Wiratma, L. (2011) Median trajectories using well-visited regions and shortest paths. *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, Chicago, USA, 1-4 November, pp. 241–250. ACM, New York.

[5] Lionov (2002) Animasi algoritma sweepline untuk membangun diagram voronoi. Skripsi. Universitas Katolik Parahyangan, Indonesia.

[6] Wiratma, L. (2010) Following the majority: a new algorithm for computing a median trajectory. Thesis. Utrecht University, The Netherlands.

[7] Wiratma, L. (2022) Coming Not Too Soon, Later, Delay, Someday, Hopefully. Disertasi. Utrecht University, The Netherlands.

[8] van kreveld, M., van Lankveld, T., dan Veltkamp, R. (2013) Watertight scenes from urban lidar and planar surfaces. Technical Report UU-CS-2013-007. Utrecht University, The Netherlands.

[9] Rekhter, Y. dan Li, T. (1994) A border gateway protocol 4 (bgp-4). RFC 1654. RFC Editor, http://www.rfc-editor.org.

[10] ITU-T Z.500 (1997) *Framework on formal methods in conformance testing*. International Telecommunications Union. Geneva, Switzerland.

[11] Version 9.0.0 (2016) *The Unicode Standard*. The Unicode Consortium. Mountain View, USA.

[12] Version 7.0 Nougat (2016) *Android API Reference Manual*. Google dan Open Handset Alliance. Mountain View, USA.

[13] Webb, R., Daruca, O., dan Alfadian, P. (2012) *Method of optimizing a text massage communication between a server and a secure element*. Paten no. EP2479956 (A1). European Patent Organisation. Munich, Germany.

[14] Wiratma, L. (2009) Median trajectory. Report for GMT Experimentation Project at Utrecht University.

[15] Lionov (2011) Polymorphism pada C++. Catatan kuliah AKS341 Pemrograman Sistem di Universitas Katolik Parahyangan, Bandung. http://tinyurl.com/lionov. 30 September 2016.

[16] Erickson, J. (2003) CG models of computation? http://www.computational-geometry.org/mailing-lists/compgeom-announce/2003-December/000852.html. 30 September 2016.

[17] AGUNG (2012) Menjajal tango 12. Majalah HAI no 02, Januari 2012.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```c

// This does not make algorithmic sense,
// but it shows off significant programming characters.

#include<stdio.h>

void myFunction( int input, float* output ) {
    switch ( array[i] ) {
        case 1: // This is silly code
            if ( a >= 0 || b <= 3 && c != x )
                *output += 0.005 + 20050;
            char = 'g';
            b = 2^n + ~right_size - leftSize * MAX_SIZE;
            c = (--aaa + &daa) / (bbb++ - ccc % 2 );
            strcpy(a,"hello $@?");
    }
    count = ~mask | 0x00FF00AA;
}

// Fonts for Displaying Program Code in LATEX
// Adrian P. Robson, nepsweb.co.uk
// 8 October 2012
// http://nepsweb.co.uk/docs/progfonts.pdf
```

Kode A.2: MyCode.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;

//class for set of vertices close to furthest edge
public class MyFurSet {
    protected int id;                                   //id of the set
    protected MyEdge FurthestEdge;                      //the furthest edge
    protected HashSet<MyVertex> set;                    //set of vertices close to furthest edge
    protected ArrayList<ArrayList<Integer>> ordered;    //list of all vertices in the set for each trajectory
    protected ArrayList<Integer> closeID;               //store the ID of all vertices
    protected ArrayList<Double> closeDist;              //store the distance of all vertices
    protected int totaltrj;                             //total trajectories in the set

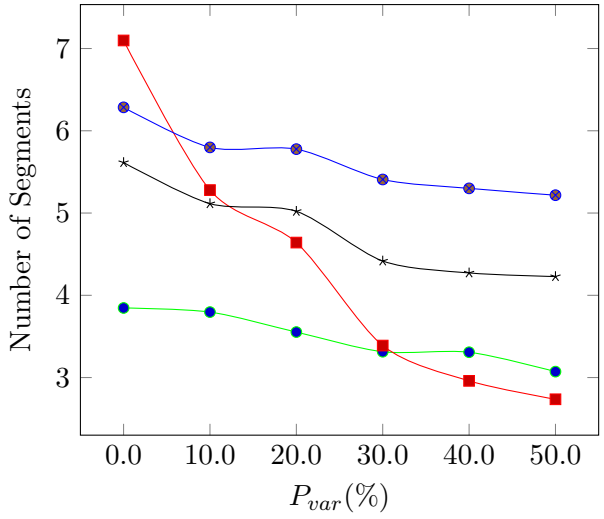
    /*
     * Constructor
     * @param id : id of the set
     * @param totaltrj : total number of trajectories in the set
     * @param FurthestEdge : the furthest edge
     */
    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
        this.id = id;
        this.totaltrj = totaltrj;
        this.FurthestEdge = FurthestEdge;
        set = new HashSet<MyVertex>();
        ordered = new ArrayList<ArrayList<Integer>>();
        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
        closeID = new ArrayList<Integer>(totaltrj);
        closeDist = new ArrayList<Double>(totaltrj);
        for (int i = 0;i <totaltrj;i++) {
            closeID.add(-1);
            closeDist.add(Double.MAX_VALUE);
        }
    }

}
```

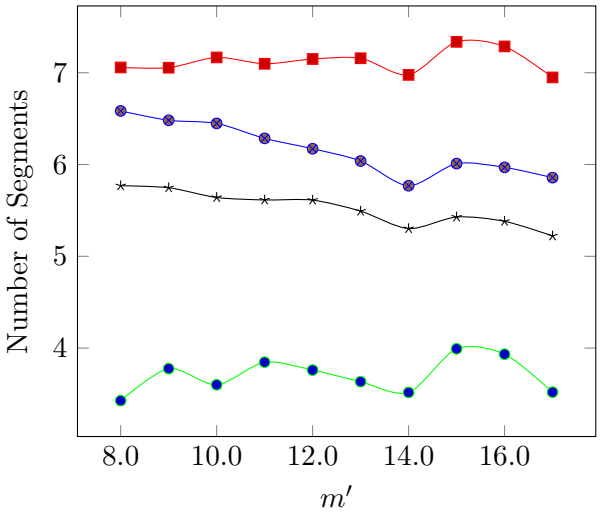
LAMPIRAN B

HASIL EKSPERIMEN

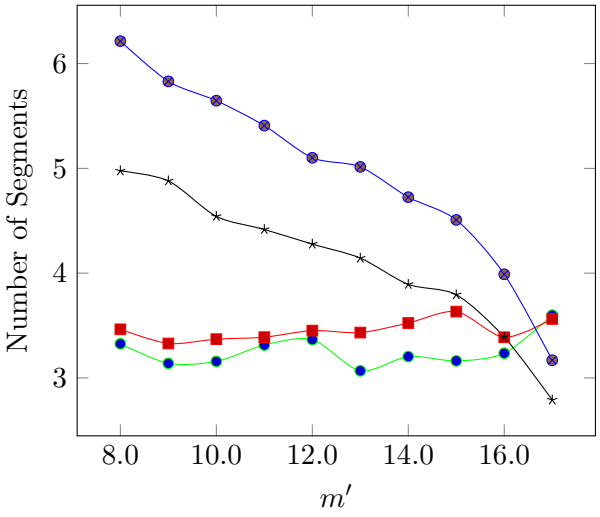
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



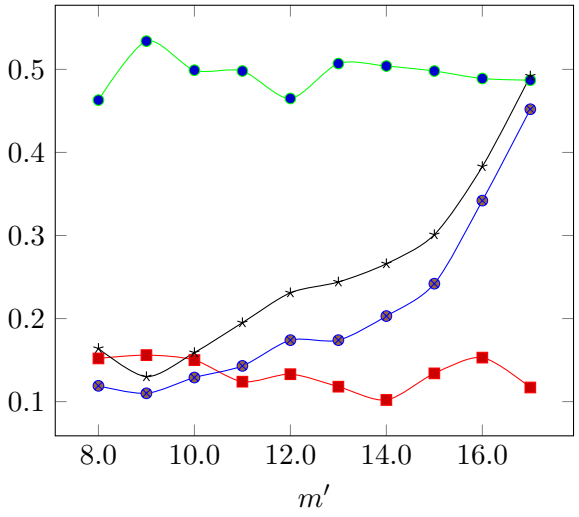
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4